

PROJECT REPORT

NOVAPLACE – PLACEMENT MANAGEMENT SYSTEM

1. TITLE AND PROJECT OVERVIEW

Project Title

NovaPlace – Placement Management System

Project Overview

NovaPlace is a web-based **Placement Management System** developed to manage student placement information efficiently. The system provides a centralized platform for storing, updating, viewing, searching, and deleting candidate placement records.

The application follows a **CRUD (Create, Read, Update, Delete)** architecture. It consists of a frontend developed using **HTML, CSS, and JavaScript** and a backend developed using **Python Django and Django REST Framework**. **SQLite** is used as the database for storing candidate information.

The system provides a user-friendly interface through which placement-related candidate information such as Candidate ID, Name, Department, CGPA, Company, Package, Email, and Placement Status can be managed.

Main Features

- User login page
- Placement dashboard
- Candidate management
- Add candidate
- View candidates
- Edit candidate details
- Delete candidate records
- Search/filter candidate records
- REST API integration
- Database storage
- Placement status management
- Responsive and professional UI

2. PROBLEM STATEMENT

Traditional placement data management often involves spreadsheets, paper records, or separate files. These methods can make it difficult to maintain accurate and updated placement information.

Some common problems include:

- Difficulty in maintaining large numbers of student records.
- Duplicate or inconsistent data.
- Time-consuming manual updates.
- Difficulty searching for a particular candidate.
- Lack of centralized placement information.
- Risk of accidentally modifying or deleting records.
- Difficulty integrating placement data with other applications.
- Lack of a proper CRUD-based management system.

Therefore, there is a need for a **centralized web-based Placement Management System** that allows placement records to be managed efficiently through a simple and organized interface.

NovaPlace addresses these issues by providing a web application with a structured database and REST API.

3. OBJECTIVES

The main objective of NovaPlace is to develop an efficient web-based system for managing placement candidate information.

Specific Objectives

1. To develop a user-friendly placement management interface.
2. To store candidate placement information in a structured database.
3. To implement **Create, Read, Update, and Delete (CRUD)** operations.
4. To provide REST API endpoints for candidate management.
5. To allow users to add new candidate records.
6. To allow existing candidate information to be updated.
7. To allow unnecessary candidate records to be deleted.
8. To provide a search and filtering facility.
9. To reduce manual placement record management.
10. To provide a scalable architecture for future enhancements.

4. TECHNOLOGY STACK

NovaPlace uses a combination of frontend, backend, database, and API technologies.

Layer	Technology
Frontend	HTML5
Styling	CSS3
Client-side Logic	JavaScript
Backend	Python
Web Framework	Django
API Framework	Django REST Framework
Database	SQLite
API Testing	Postman
Code Editor	Visual Studio Code
Version Control	Git / GitHub
Frontend Deployment	Vercel
Backend Deployment	PythonAnywhere

HTML5

HTML5 is used to create the structure of the web pages.

CSS3

CSS is used to design the interface, including:

- Layout
- Colors
- Buttons
- Forms
- Cards
- Navigation
- Responsive design

JavaScript

JavaScript provides client-side functionality and communicates with the backend API using HTTP requests.

Python Django

Django handles the backend application logic and database communication.

Django REST Framework

Django REST Framework is used to create RESTful API endpoints for CRUD operations.

SQLite

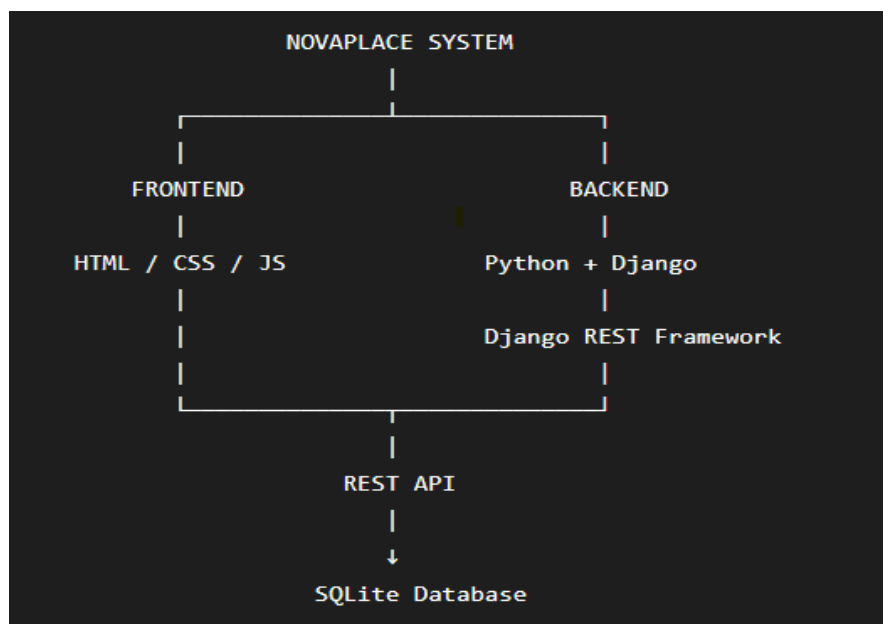
SQLite stores candidate information in a structured relational database.

Postman

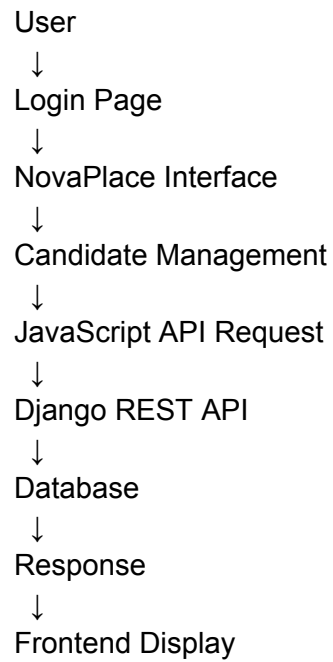
Postman is used to test the API operations independently.

5. SYSTEM ARCHITECTURE

NovaPlace follows a **three-layer web architecture**.



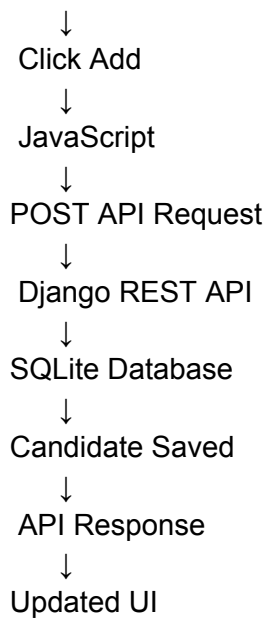
Architecture Flow



Example

When a user adds a candidate:

User enters candidate details



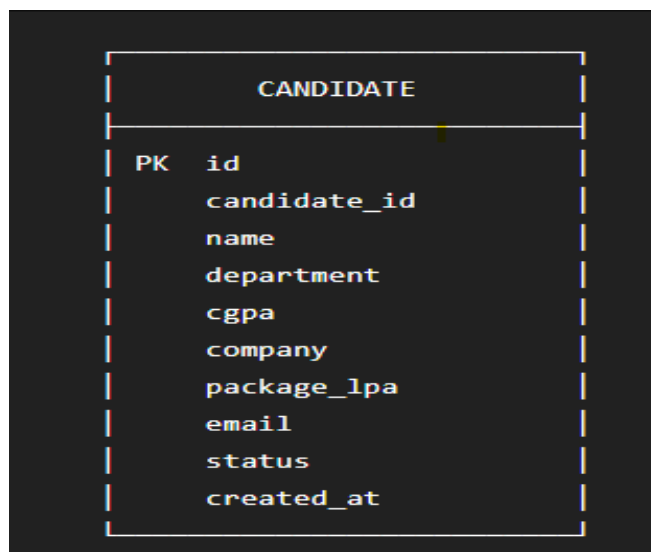
6. DATABASE / ER DIAGRAM

The main entity in the system is **Candidate**.

Candidate Entity

Attribute	Description
id	Primary key
Candidate_ id	Unique candidate identification
name	Candidate name
department	Candidate department
cgpa	Candidate CGPA
company	Placed/recruiting company
Package_ lpa	Package in LPA
email	Candidate email
status	Placement status
created_at	Record creation date/time

ER Diagram



Database Purpose

The database provides persistent storage for candidate placement information. CRUD operations performed through the REST API directly interact with the Candidate table.

7. UI SCREENSHOTS

The project contains several user interface screens.

7.1 Login Page

The login page provides:

- Email field
- Password field
- Login button
- Demo credentials
- Validation message
- NovaPlace branding

Demo credentials:

Email: demo@novaplace.com

Password: Nova@123

7.2 Landing Page

The landing page introduces NovaPlace and provides access to the main application.

7.3 Dashboard

The dashboard provides an overview of placement records and quick access to candidate management functions.

7.4 Candidate Management

The candidate management interface displays candidate records and provides options to:

- Add
- Edit
- Delete
- Search

7.5 Add Candidate Form

The form collects:

Candidate ID
Name
Department
CGPA
Company
Package
Email
Status

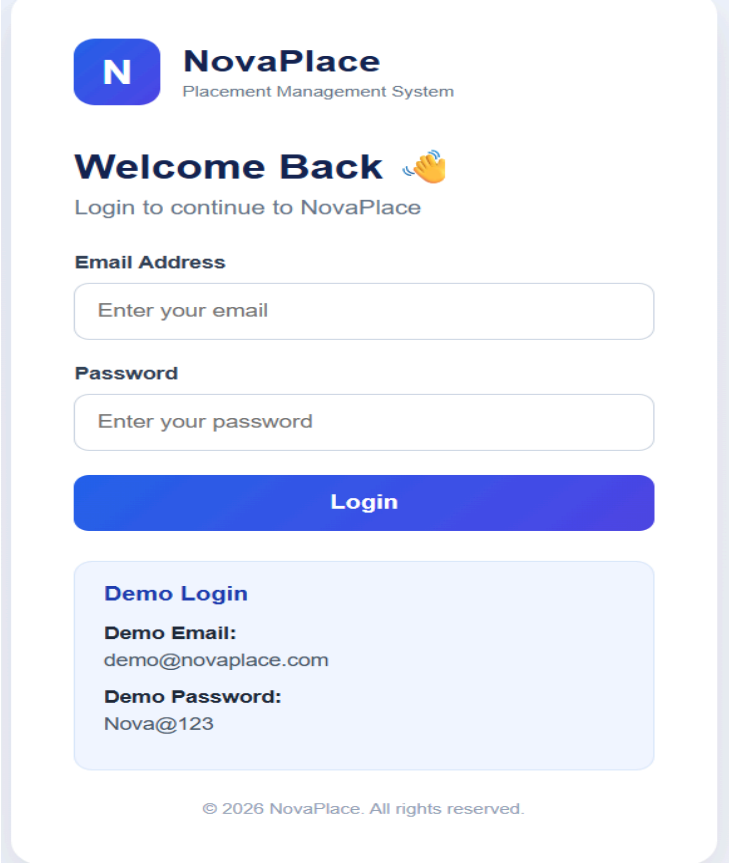
7.6 Search Interface

Users can search/filter candidate records to quickly locate relevant information.

Screenshot Section

For final report, inserted actual screenshots under these headings:

Figure 1 – NovaPlace Login Page



The screenshot displays the NovaPlace login interface. At the top left is the NovaPlace logo, consisting of a blue square with a white 'N' and the text 'NovaPlace Placement Management System' to its right. Below the logo, the text 'Welcome Back' is followed by a yellow hand icon. Underneath, it says 'Login to continue to NovaPlace'. The form includes two input fields: 'Email Address' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your password'. A blue 'Login' button is positioned below these fields. At the bottom, a light blue box contains 'Demo Login' information: 'Demo Email: demo@novaplace.com' and 'Demo Password: Nova@123'. The footer at the very bottom reads '© 2026 NovaPlace. All rights reserved.'

Figure 2 – NovaPlace Landing Page

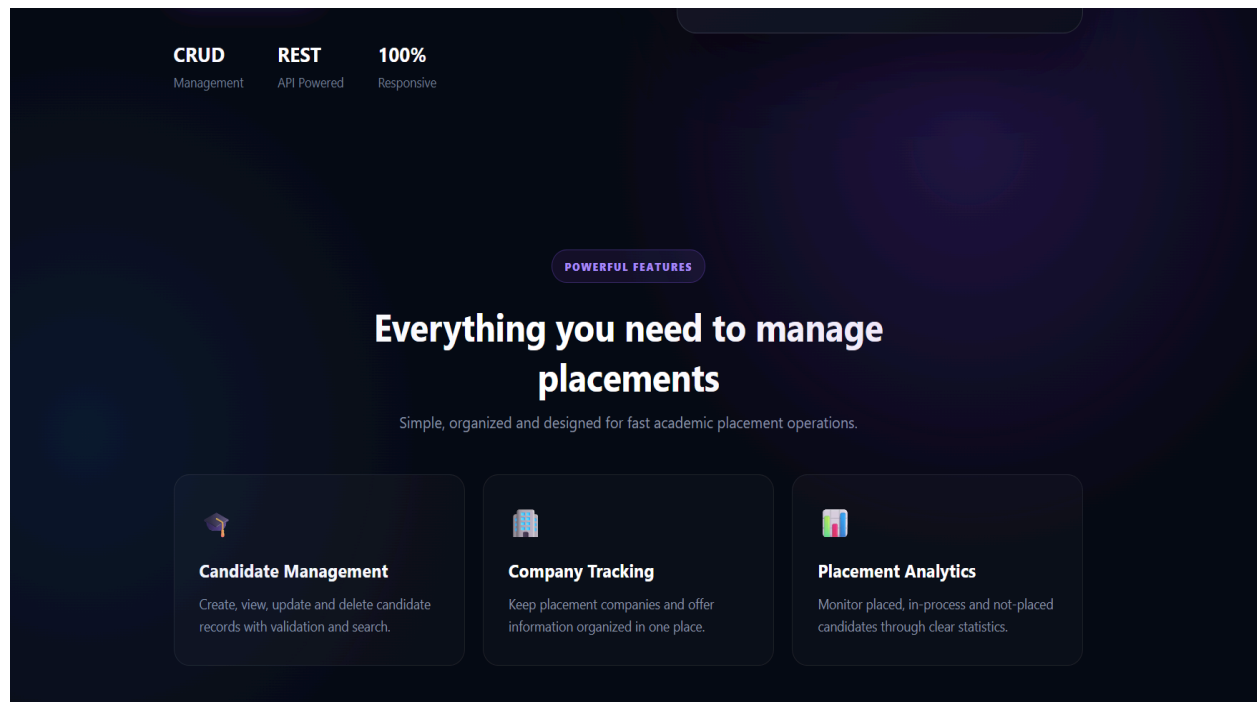
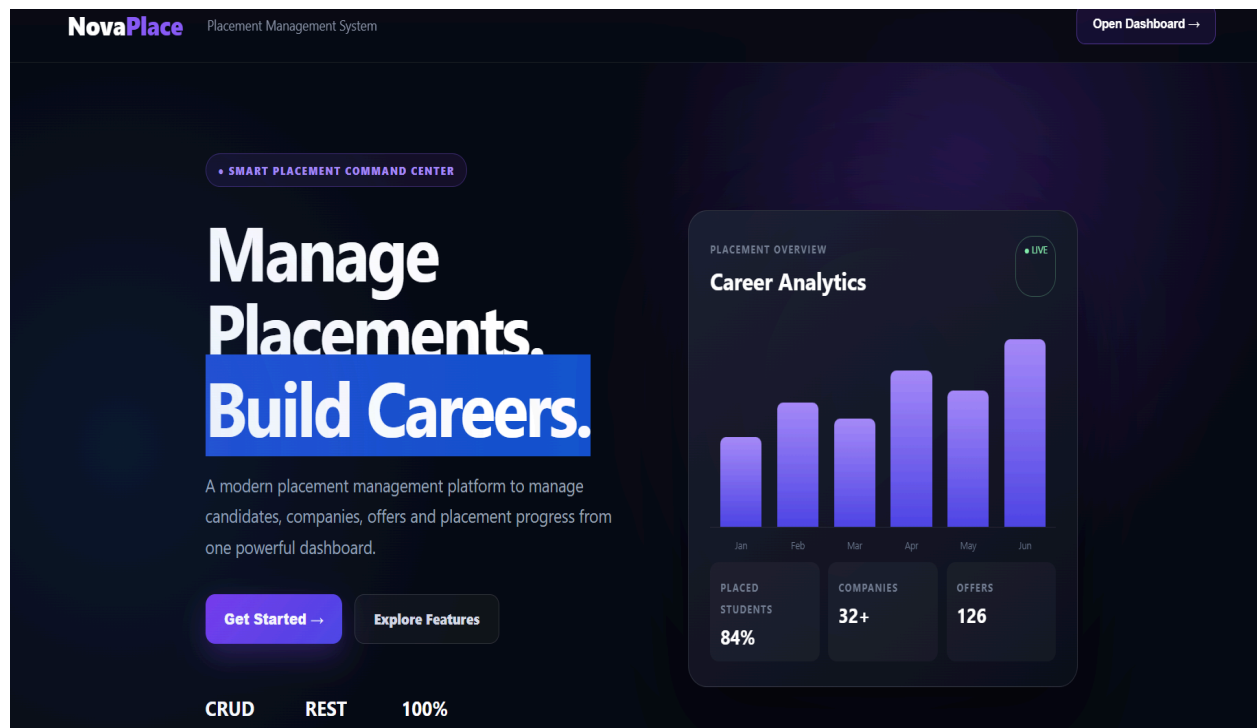


Figure 3 – NovaPlace Dashboard

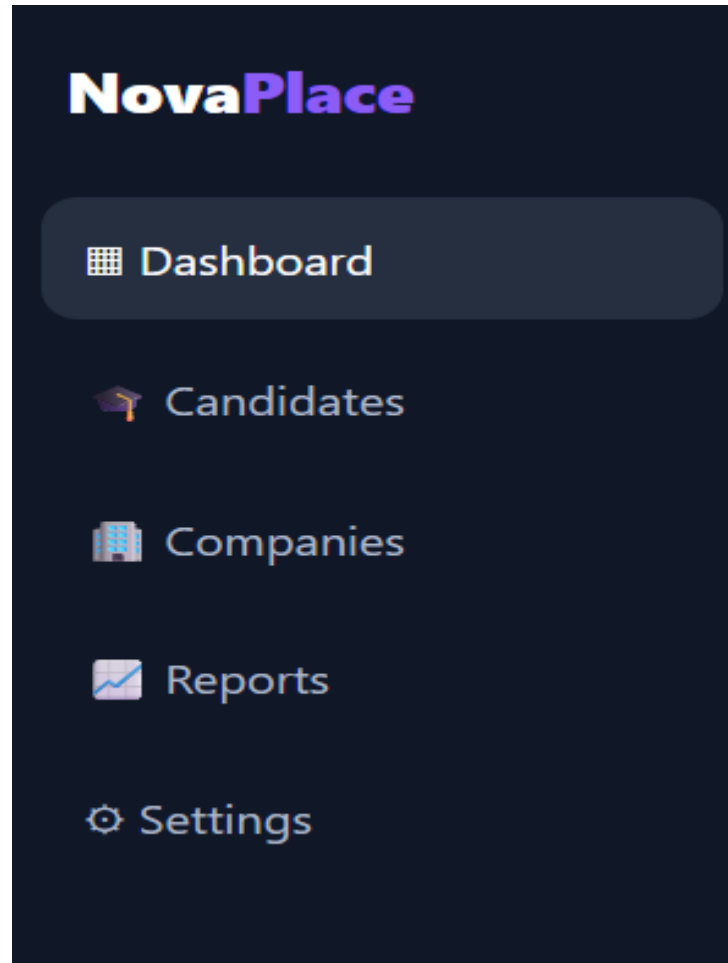


Figure 4 – Candidate Management Page

NovaPlace

Dashboard

Candidates

Companies

Reports

Settings

PLACEMENT MANAGEMENT

Career Command Center

Manage student placements, candidates and companies.

Refresh

Add Candidate

Candidates1

Placed1

In Process0

Companies1

Candidate Records

View and manage placement candidates.

Search candidates...

CANDIDATE ID	NAME	DEPARTMENT	CGPA	COMPANY	PACKAGE	STATUS	ACTIONS
105	Arun Kumar	IT	8.75	TCS	8.50 LPA	Placed	Edit Delete

Figure 5 – Add Candidate Form

CREATE

Add Candidate

X

Candidate ID *

Full Name *

Department *

CGPA *

Company *

Package (LPA) *

Email *

Status *

In Process

Cancel

Create Candidate

Figure 6 – Search Candidate Page

Candidate Records

View and manage placement candidates.

105

CANDIDATE ID	NAME	DEPARTMENT	CGPA	COMPANY	PACKAGE	STATUS	ACTIONS
105	Arun Kumar	IT	8.75	TCS	8.50 LPA	Placed	EditDelete

Figure 7 – Edit Candidate Page

UPDATE

Edit Candidate

X

Candidate ID *

105

Full Name *

Riya

Department *

IT

CGPA *

8.75

Company *

TCS

Package (LPA) *

8.50

Email *

riya@gmail.com

Status *

Placed

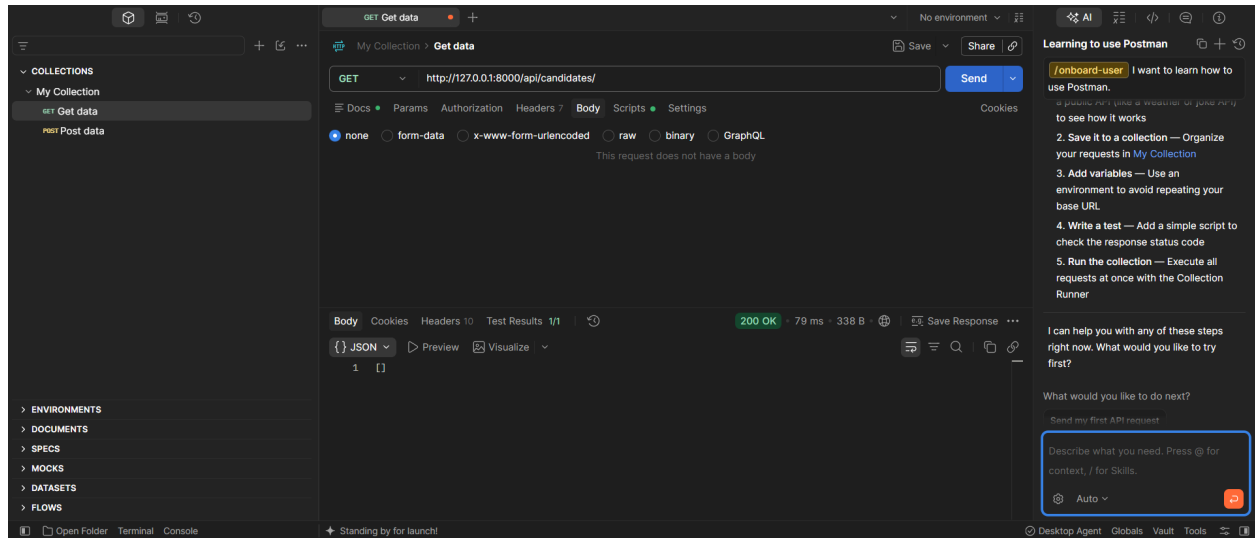
▼

Cancel

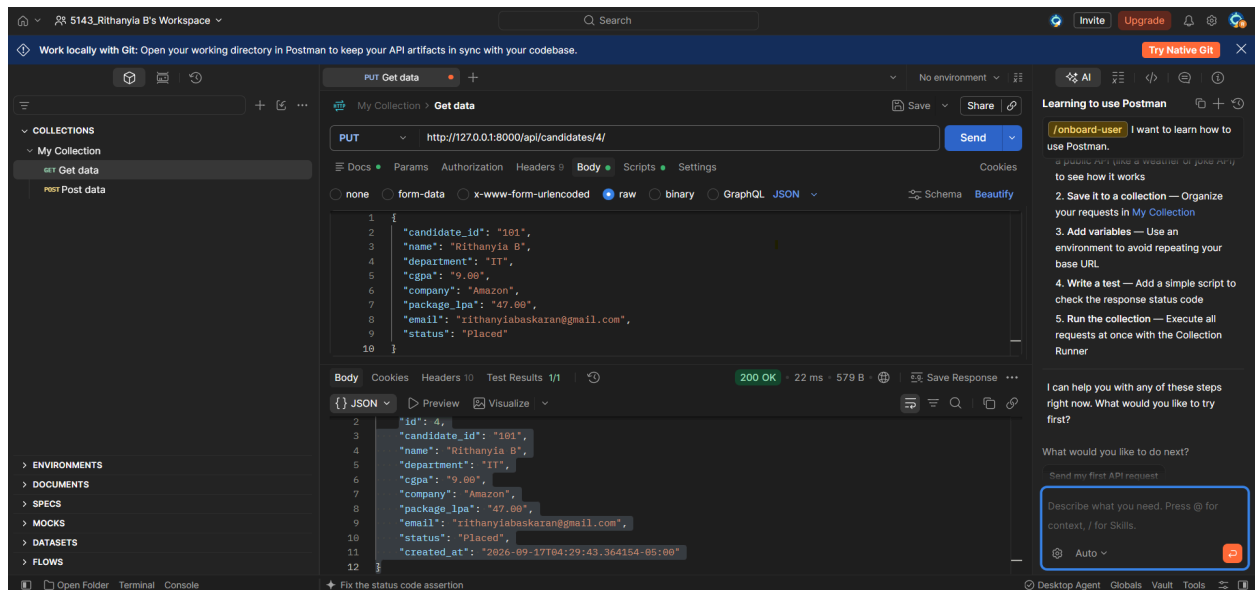
Save Changes

Figure 8 – Postman API Testing

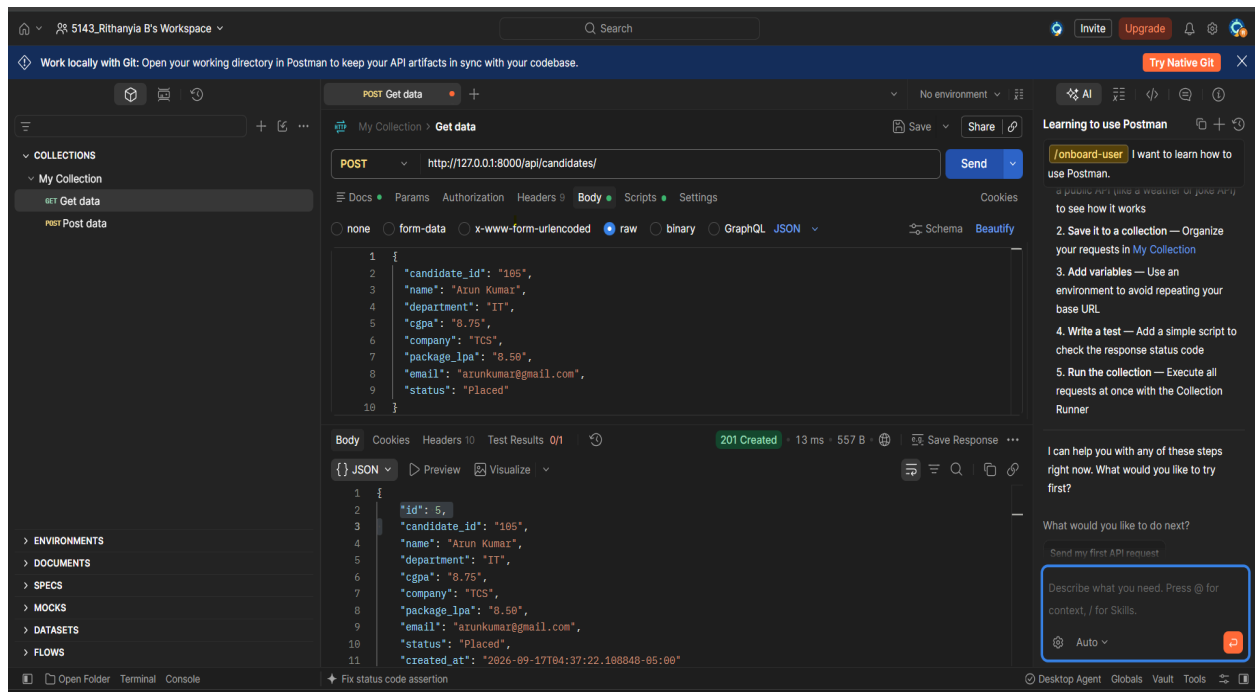
GET



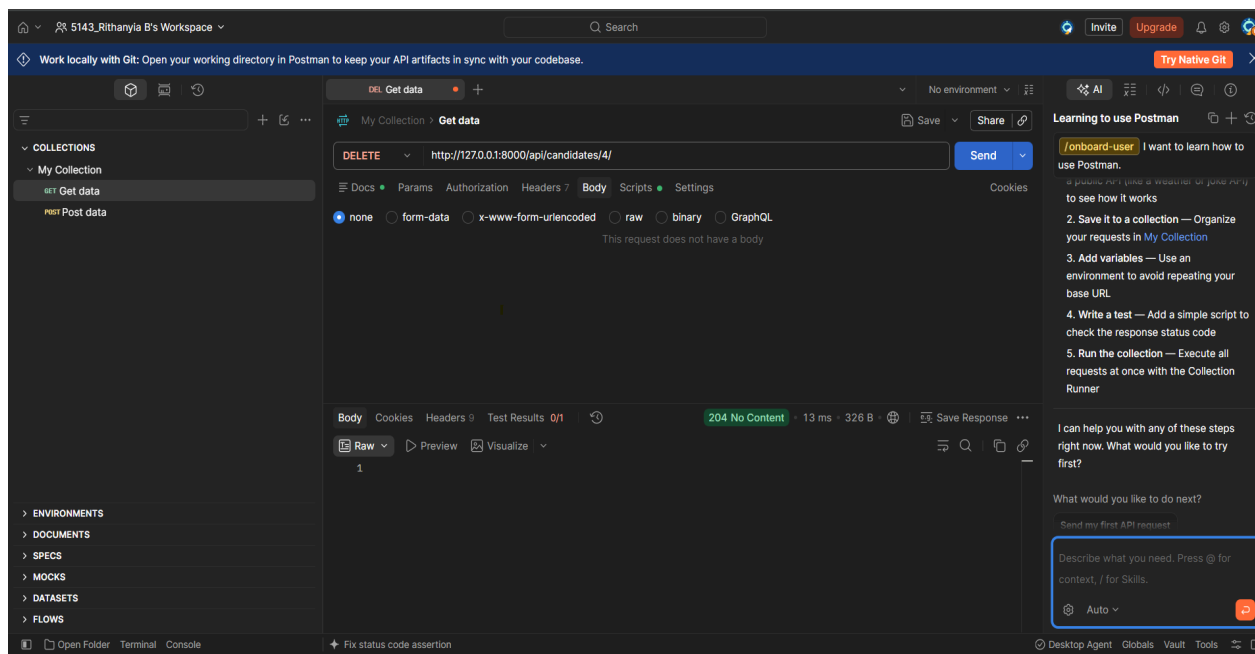
PUT



POST



DELETE



8. API ENDPOINT DOCUMENTATION

The NovaPlace backend provides REST API endpoints for candidate management.

Base URL – Local Development

`http://127.0.0.1:8000/api/candidates/`

API Operations

Operation	Method	Endpoint
Get all candidates	GET	<code>/api/candidates/</code>
Add candidate	POST	<code>/api/candidates/</code>
Get one candidate	GET	<code>/api/candidates/{id}/</code>
Update candidate	PUT	<code>/api/candidates/{id}/</code>
Delete candidate	DELETE	<code>/api/candidates/{id}/</code>

GET – Retrieve Candidates

Endpoint

GET `/api/candidates/`

Purpose

Retrieves all candidate records from the database.

Example:

```
{
  "candidate_id": "101",
  "name": "Riya S ",
  "department": "IT",
  "cgpa": "9.00",
```

```
"company": "Amazon",  
"package_lpa": "47.00",  
"email": "riya83@gmail.com",  
"status": "Placed"  
}
```

9. CRUD IMPLEMENTATION DETAILS

CRUD represents:

C → Create

R → Read

U → Update

D → Delete

9.1 CREATE

The CREATE operation is implemented using the **POST** method.

Endpoint:

POST /api/candidates/

Example:

```
{  
  "candidate_id": "105",  
  "name": "Rita M",  
  "department": "IT",  
  "cgpa": "8.75",  
  "company": "TCS",  
  "package_lpa": "8.50",  
  "email": "rita87@gmail.com",  
  "status": "Placed"  
}
```

The API validates the information and stores the new candidate in the database.

9.2 READ

The READ operation uses the **GET** method.

All Candidates

GET /api/candidates/

Individual Candidate

GET /api/candidates/4/

The server retrieves the requested candidate information and returns it as JSON.

9.3 UPDATE

The UPDATE operation uses the **PUT** method.

Example:

PUT /api/candidates/4/

Example request:

```
{
  "candidate_id": "101",
  "name": "Riya S ",
  "department": "IT",
  "cgpa": "9.56",
  "company": "Amazon",
  "package_lpa": "47.00",
  "email": "riya83@gmail.com",
  "status": "Placed"
}
```

The existing candidate record is modified with the new values.

9.4 DELETE

The DELETE operation uses the **DELETE** method.

Example:

DELETE /api/candidates/4/

The server removes the candidate record corresponding to database ID 4.

10. TESTING RESULTS

The application was tested using both the frontend interface and Postman.

API Testing

Test Case	Method	Expected Result	Result
Retrieve all candidates	GET	Candidate list returned	Pass
Retrieve one candidate	GET	Candidate details returned	Pass
Add candidate	POST	New candidate created	Pass
Update candidate	PUT	Candidate updated	Pass
Delete candidate	DELETE	Candidate deleted	Pass
Invalid candidate ID	GET	Error response returned	Pass
Empty POST request	POST	Validation error	Pass

Example Successful POST

A candidate was added through Postman using:

POST
http://127.0.0.1:8000/api/candidates/

The server returned the newly created candidate with a database ID.

Example DELETE

A candidate with database ID 4 was deleted using:

DELETE

<http://127.0.0.1:8000/api/candidates/4/>

The deletion was reflected when the candidate list was retrieved again.

11. INSTALLATION AND EXECUTION STEPS

Requirements

The following software is required:

- Python
- Django
- Django REST Framework
- Visual Studio Code
- Web browser
- Postman

Backend Installation

Open the terminal and navigate to:

```
cd
C:\Users\Name\Downloads\NovaCRUD_Placement_Management\NovaCRUD_Placement_Management\backend
```

Install required packages:

```
python -m pip install django djangorestframework django-cors-headers
```

Run database migrations:

```
python manage.py migrate
```

Check the project:

```
python manage.py check
```

Start the backend:

```
python manage.py runserver
```

Backend URL:

```
http://127.0.0.1:8000/
```

API:

```
http://127.0.0.1:8000/api/candidates/
```

Frontend Execution

Open another terminal:

```
cd  
C:\Users\Name\Downloads\NovaCRUD_Placement_Management\NovaCRUD_Placement_Management\frontend
```

Run:

```
py -m http.server 5500
```

Open:

```
http://127.0.0.1:5500/login.html
```

The frontend communicates with the Django REST API.

12. CHALLENGES AND SOLUTIONS

Challenge 1 – REST Framework Module Error

Problem:

ModuleNotFoundError: No module named 'rest_framework'

Solution:

Django REST Framework was installed using:

```
python -m pip install djangorestframework
```

Challenge 2 – Frontend and Backend Communication

The frontend and backend were running on different ports.

Frontend:

127.0.0.1:5500

Backend:

127.0.0.1:8000

This can result in cross-origin request restrictions.

Solution:

`django-cors-headers` was added and configured in Django.

Challenge 3 – POST Request Returning 400

An empty POST request resulted in:

```
{  
  "detail": "..."  
}
```

Solution:

A valid JSON request body was provided with all required candidate fields.

Challenge 4 – Candidate Not Found

The following error was encountered:

```
{  
  "detail": "No Candidate matches the given query."  
}
```

Solution:

The correct database `id` was identified using the GET endpoint.

For example:

GET `/api/candidates/`

If the database ID is `4`, the individual endpoint becomes:

`/api/candidates/4/`

Challenge 5 – Localhost API During Deployment

The frontend originally used:

```
const API = "http://127.0.0.1:8000/api/candidates/";
```

This works only during local development.

Solution:

During deployment, the localhost URL must be replaced with the public backend URL.

13. FUTURE ENHANCEMENTS

Several improvements can be added to NovaPlace in the future.

1. Secure Authentication

A complete Django authentication system can replace the current demonstration login.

2. Role-Based Access

Different access levels can be provided for:

- Administrator
- Placement Officer
- Student

3. PostgreSQL Database

SQLite can be replaced with PostgreSQL for a larger production deployment.

4. Advanced Search

Search can be extended to include:

- Department
- Company
- CGPA
- Placement status
- Package range

5. Analytics Dashboard

Charts can display:

- Total candidates
- Total placed students
- Company-wise placement
- Department-wise placement
- Average package

6. Export Reports

Candidate records can be exported as:

- PDF
- Excel
- CSV

7. Email Notifications

The system can send notifications about placement updates and recruitment activities.

8. Cloud Database

A cloud-hosted database can provide improved persistence and scalability.

9. Mobile Responsive Improvements

The system can be optimized further for mobile and tablet devices.

10. Production Security

Future production deployment should include:

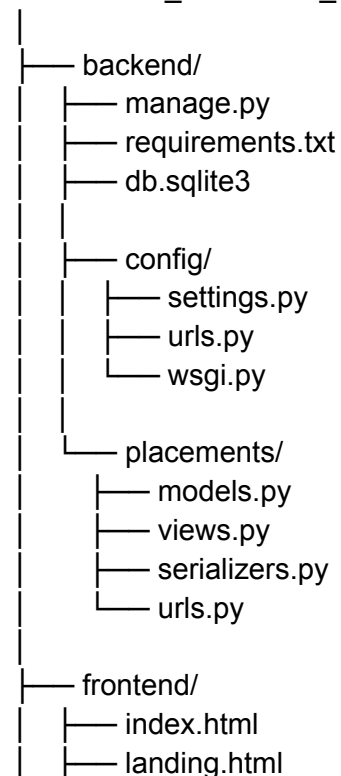
- HTTPS
- Restricted CORS
- Secure secret keys
- Proper `ALLOWED_HOSTS`
- Authentication
- Database backups

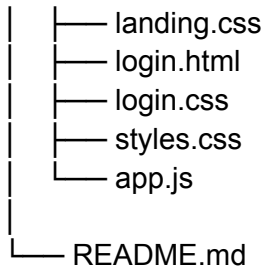
14. GIT REPOSITORY / REFERENCE DETAILS

The project source code can be maintained using **GitHub** for version control and collaboration.

Repository Structure

NovaCRUD_Placement_Management/





15. CONCLUSION

NovaPlace – Placement Management System provides a centralized and user-friendly solution for managing candidate placement information.

The project successfully implements the four fundamental **CRUD operations** using a Django REST API and SQLite database. The HTML, CSS, and JavaScript frontend provides an organized interface for interacting with the backend.

The integration of **Django REST Framework and Postman** makes it possible to develop and test the API independently. The modular architecture also provides a foundation for future features such as authentication, analytics, cloud databases, report generation, and role-based access.

Overall, NovaPlace demonstrates how a modern web application can be developed using **HTML, CSS, JavaScript, Python Django, Django REST Framework, SQLite, and REST APIs** to simplify placement record management.